

Computer Arithmetic

Session 4: Real Arithmetic

Table of Contents:

1

Floating Point Representation

4

Floating-Point Multipliers/Dividers

2

Rounding and Exceptions

5

Fused Multiply-Add

3

Floating-Point Adders/Subtractors

6

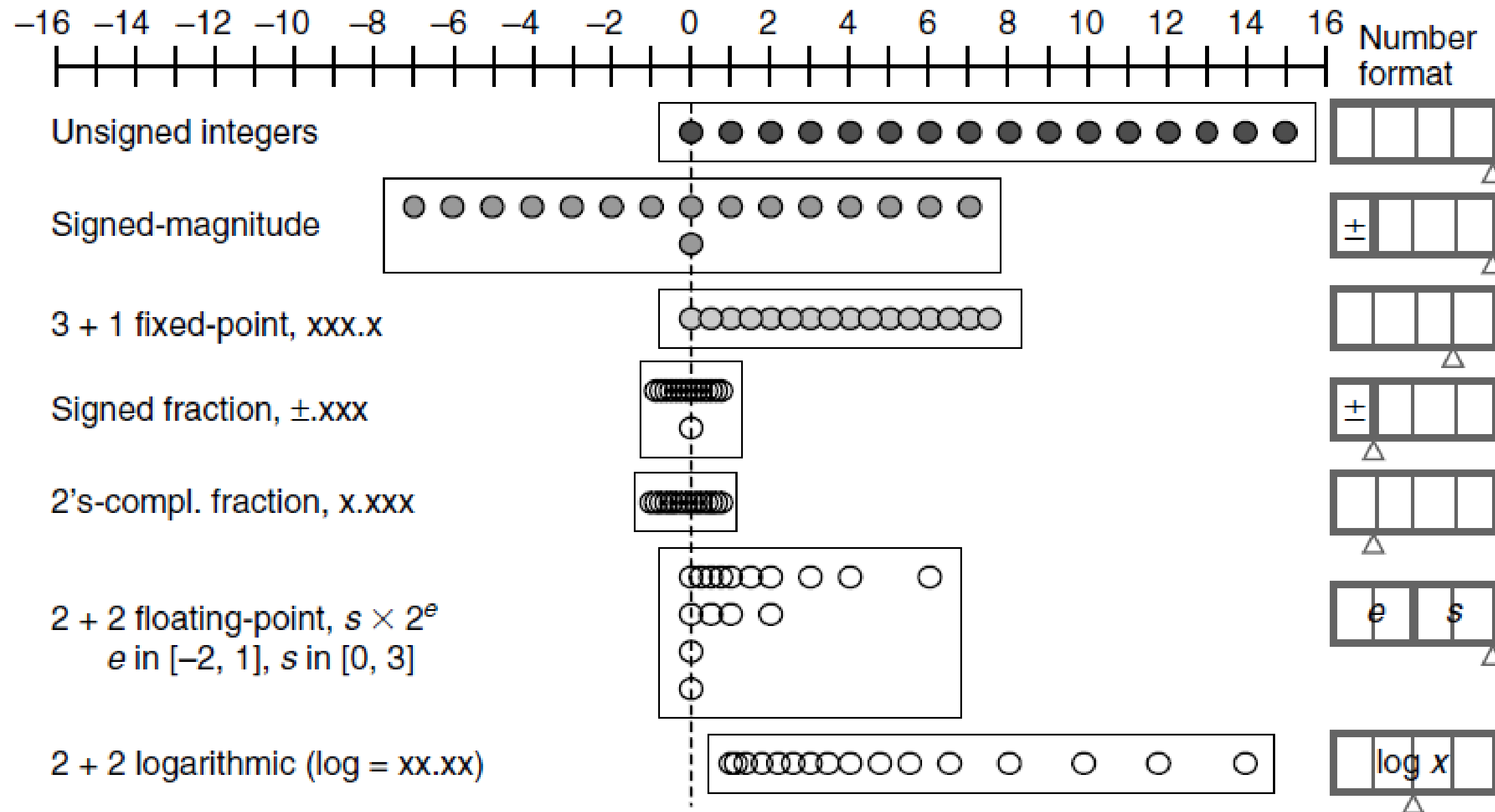
Logarithmic Arithmetic Unit

1

Floating Point Representation



Floating Point Representation



Floating Point Representation

Why is floating-point needed anyway?

Fixed-point representation has 2 main problems:

- Limited Dynamic Range.
- Limited Precision.

Assume that we have 2 U4.4 fixed-point numbers:

4.25: 0100.0100

6.5: 0110.1000

Multiplying these 2 numbers yields 27.625, which is already outside the range of representable values.

Floating Point Representation

Why is floating-point needed anyway?

If we wish to represent large numbers, a large number of bits is needed, which in turn incurs significant hardware and timing overhead.

Here, the floating-point representation comes into play, with its ability to represent very large or very small numbers by changing our point of view on how they are represented.

Which utilizes the scientific notation for representing numbers, where we are only required to store the fractional part and the exponent of the number and its sign.

Floating Point Representation

	Sign	Biased exponent	Significand $s = 1.f$ (the 1 is hidden)
	$\pm$	$e + \text{bias}$	f
16-bit:		5 bits, bias = 15	10 + 1 bits, half precision format
32-bit:		8 bits, bias = 127	23 + 1 bits, single-precision or short format
64-bit:		11 bits, bias = 1023	52 + 1 bits, double-precision or long format
128-bit:		15 bits, bias = 16 383	112 + 1 bits, quadruple-precision format

Figure 17.3 The IEEE 754-2008 binary floating-point number representation formats.

Floating Point Representation

Table 17.1 Some features of the IEEE 754-2008 short and long floating-point formats

Feature	Single/Short	Double/Long
Word width, bits	32	64
Significand bits	23 + 1 hidden	52 + 1 hidden
Significand range	$[1, 2 - 2^{-23}]$	$[1, 2 - 2^{-52}]$
Exponent bits	8	11
Exponent bias	127	1023
Zero (± 0)	$e + bias = 0, f = 0$	$e + bias = 0, f = 0$
Subnormal	$e + bias = 0, f \neq 0$ represents $\pm 0.f \times 2^{-126}$	$e + bias = 0, f \neq 0$ represents $\pm 0.f \times 2^{-1022}$
Infinity ($\pm \infty$)	$e + bias = 255, f = 0$	$e + bias = 2047, f = 0$
Not-a-number (NaN)	$e + bias = 255, f \neq 0$	$e + bias = 2047, f \neq 0$
Ordinary number	$e + bias \in [1, 254]$ $e \in [-126, 127]$ represents $1.f \times 2^e$	$e + bias \in [1, 2046]$ $e \in [-1022, 1023]$ represents $1.f \times 2^e$
<i>min</i>	$2^{-126} \approx 1.2 \times 10^{-38}$	$2^{-1022} \approx 2.2 \times 10^{-308}$
<i>max</i>	$\approx 2^{128} \approx 3.4 \times 10^{38}$	$\approx 2^{1024} \approx 1.8 \times 10^{308}$

Floating Point Representation

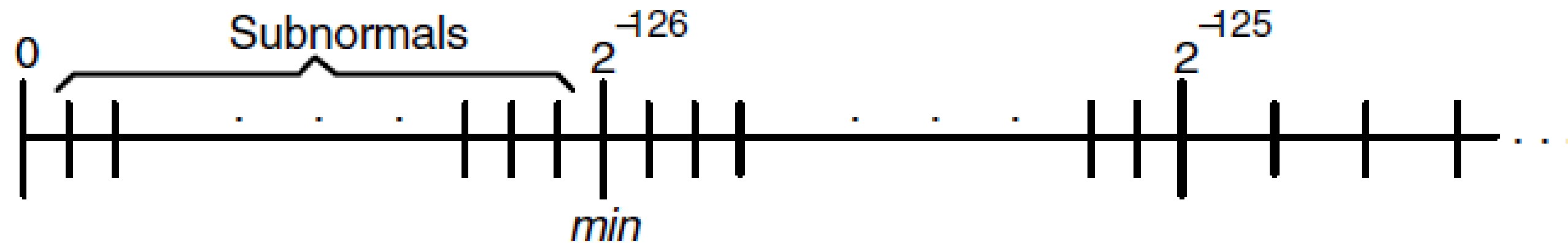


Figure 17.4 Subnormals in the IEEE single-precision format.

2

Rounding and **Exceptions**



Rounding and Exceptions

Floating-point representation is, by definition, not exact. Therefore, we will have to resort to rounding to obtain as close an answer as the exact solution.

In the following section, we will explore different rounding schemes and pick the one most suitable for our case.

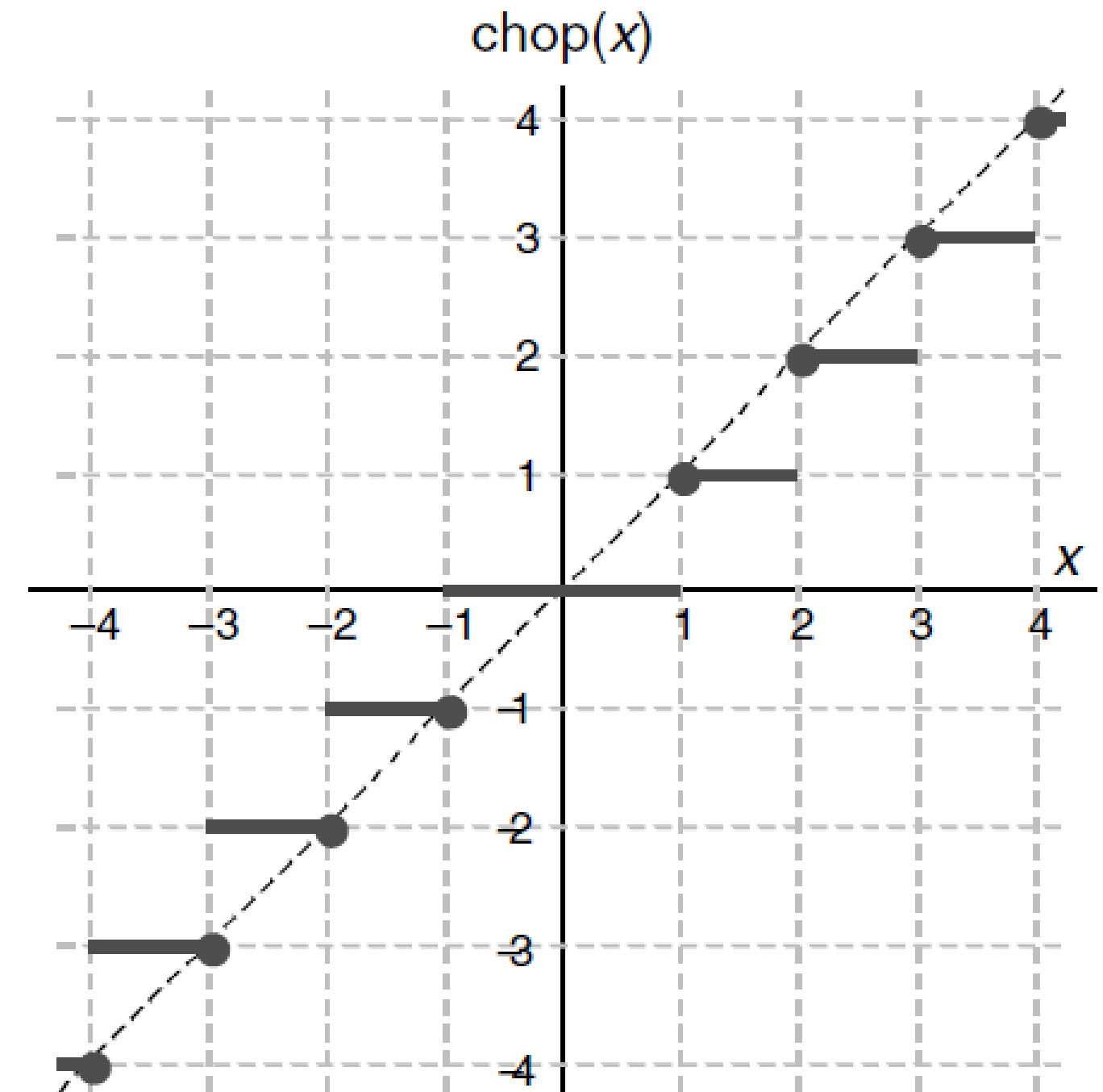
$$x_{k-1}x_{k-2} \cdots x_1x_0.x_{-1}x_{-2} \cdots x_{-l} \xrightarrow{\text{round}} y_{k-1}y_{k-2} \cdots y_1y_0.$$

Rounding and Exceptions

Chopping/Truncation:

Signed Magnitude Number:
Chopping reduces the magnitude of the number.

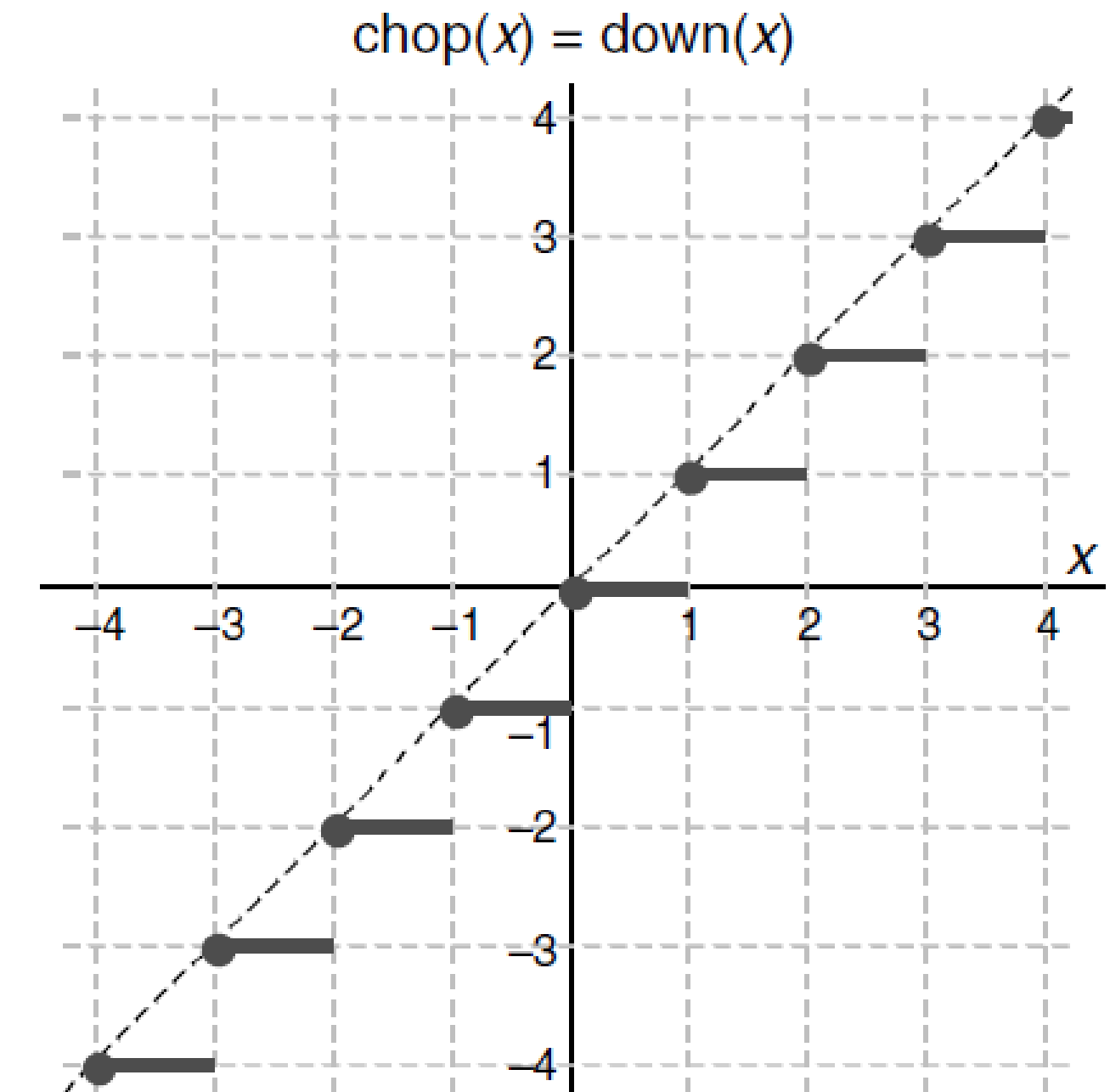
So, this method is actually
equivalent to rounding towards
zero.



Rounding and Exceptions

Chopping/Truncation:

2s Complement Number:
Chopping always reduces the
value of the number.
Which is equivalent to rounding
towards -infinity.

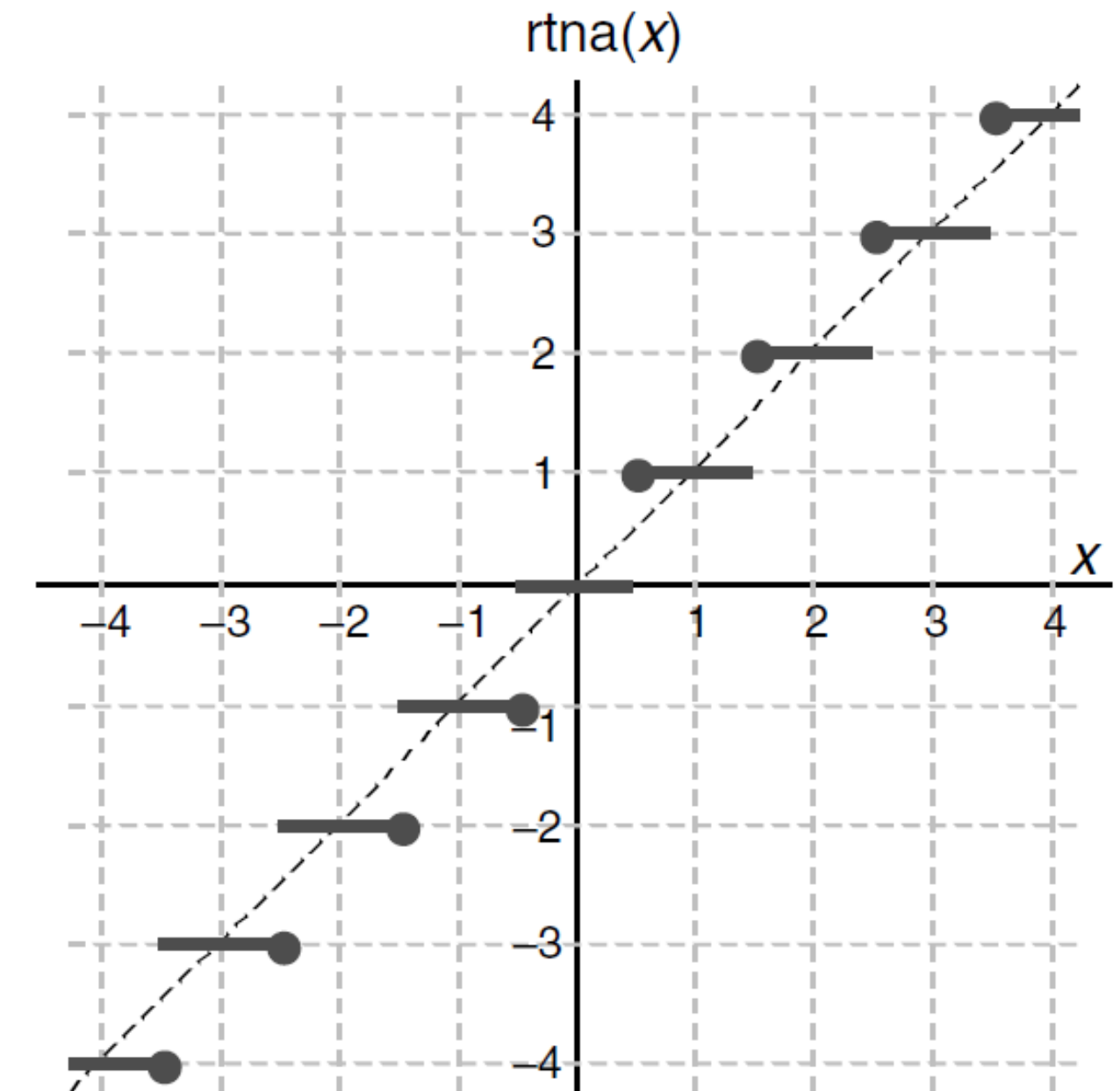


Rounding and Exceptions

Rounding to the nearest number:

The standard rounding scheme is to round down if the fractional part is less than half, and to round up.

What do all these schemes have in common?



Rounding and Exceptions

Average Error:

$x_{-1}x_{-2} = 00$	Round down	error = 0
$x_{-1}x_{-2} = 01$	Round down	error = -0.25
$x_{-1}x_{-2} = 10$	Round up	error = 0.5
$x_{-1}x_{-2} = 11$	Round up	error = 0.25

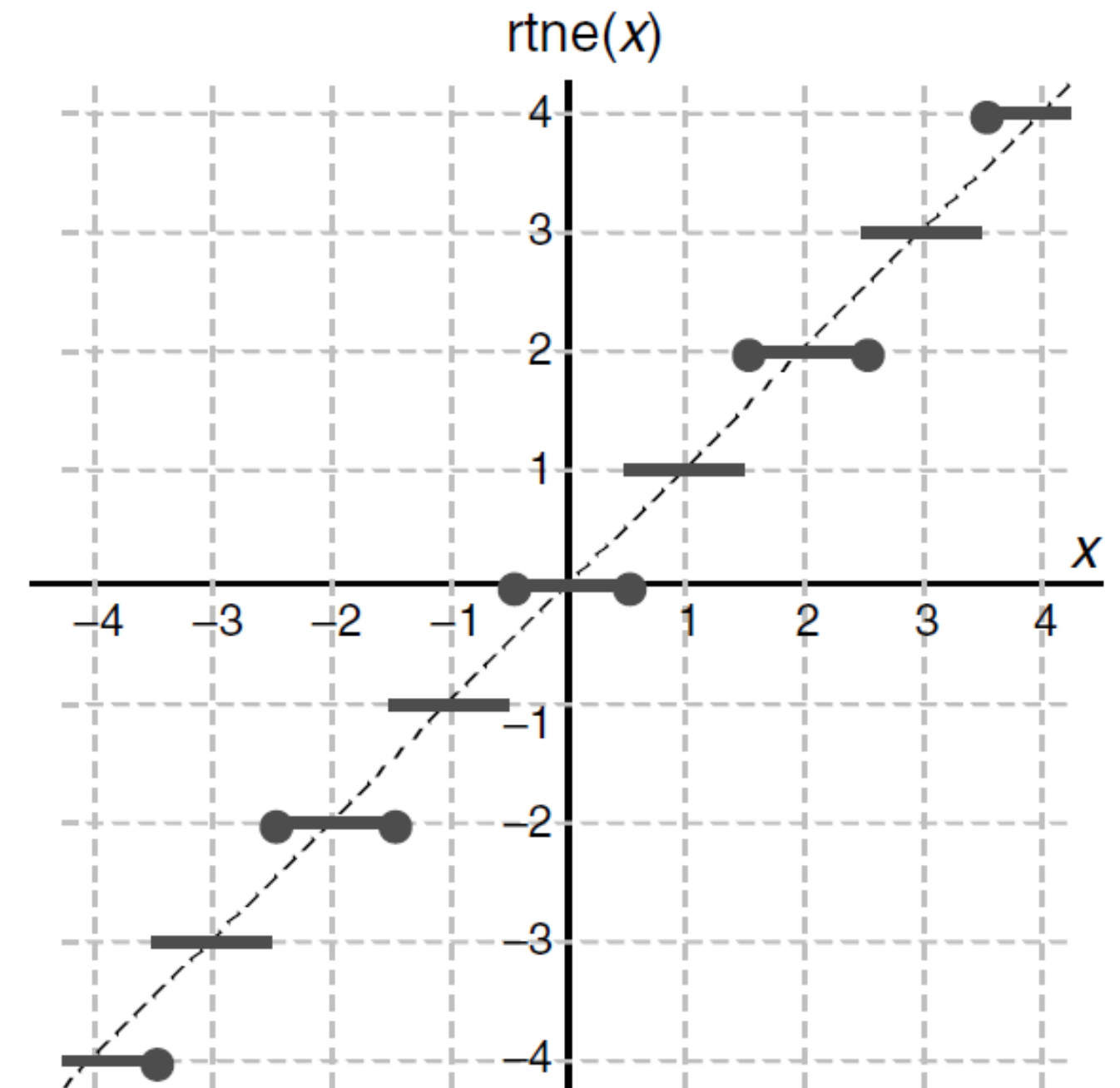
If these four cases occur with equal probability, the average error is 0.125.

This means that the rounding scheme is "Biased".
The resulting bias may create problems owing to error accumulation.

Rounding and Exceptions

Rounding to the nearest even:

The catch in this scheme is that there is no bias whatsoever, since the rounding is symmetric around each number. How?



Real Arithmetic

Rounding and Exceptions

Rounding to the nearest even:

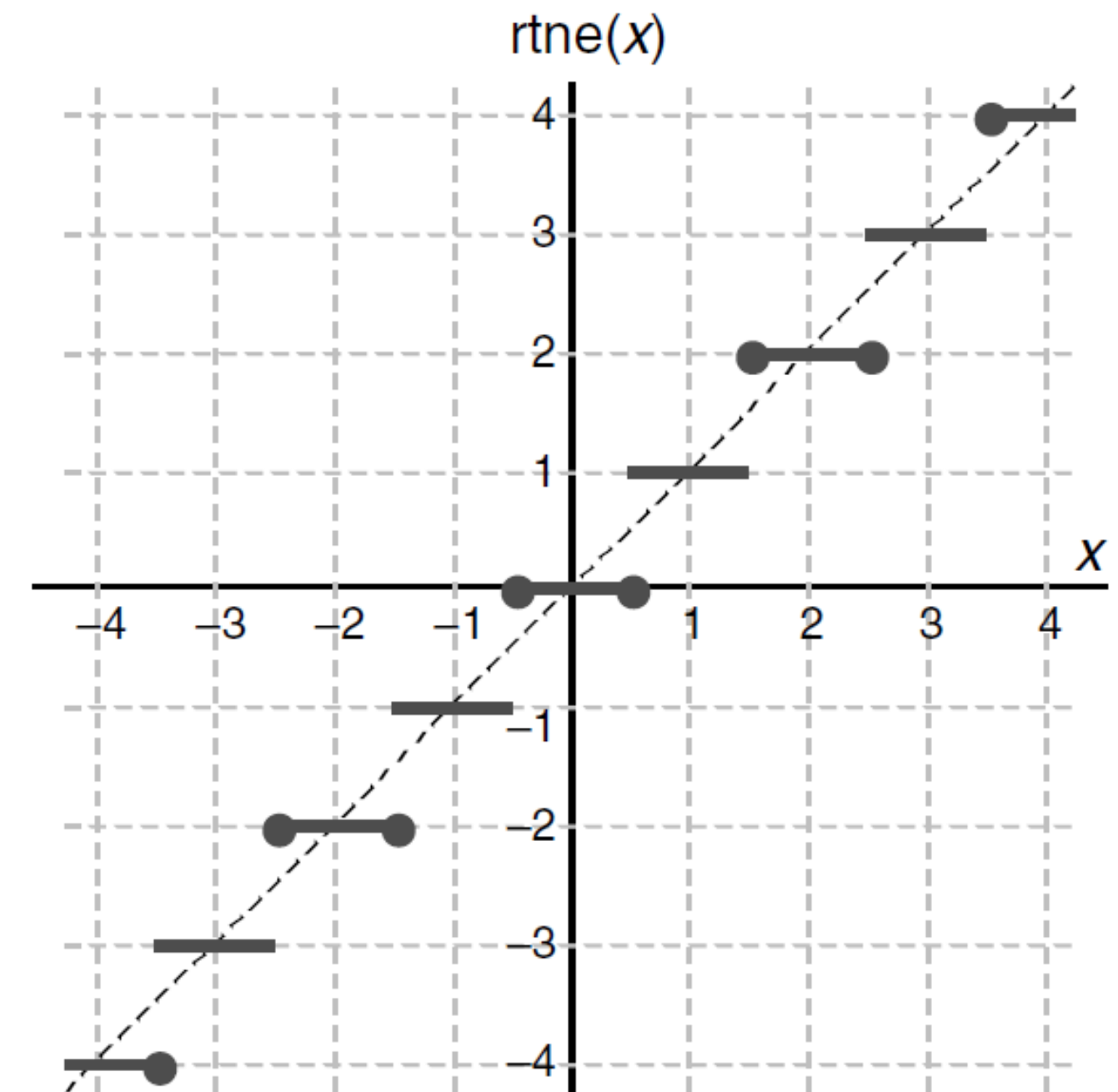
Example: around the number 2,
the range we are interested in is
between 1.5 and 2.5.

2.5 $\rightarrow$ -0.5 error

1.5 $\rightarrow$ 0.5 error

If the probability of each value is
half:

$(0.5 - 0.5) / 2 = 0 \rightarrow$ **unbiased.**



Rounding and Exceptions

Exceptions:

Some special cases need to be handled individually and are encoded to be recognized by the arithmetic unit:

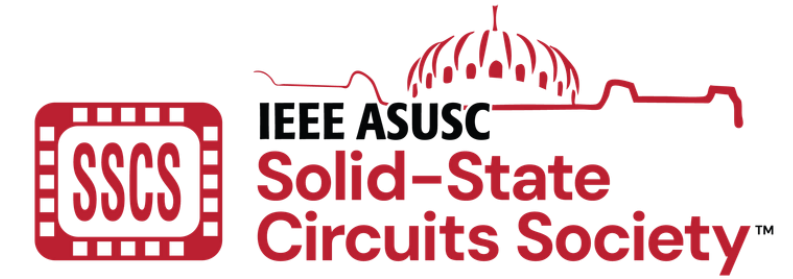
- Zero: 0 00000000 000000000000000000000000
- Infinity: 0 11111111 000000000000000000000000
- qNaN: 0 11111111 100000000000000000000000
- sNaN: 0 11111111 010000000000000000000000

3

Floating-Point **Adders/Subtractors**



Floating-Point Adders/Subtractors

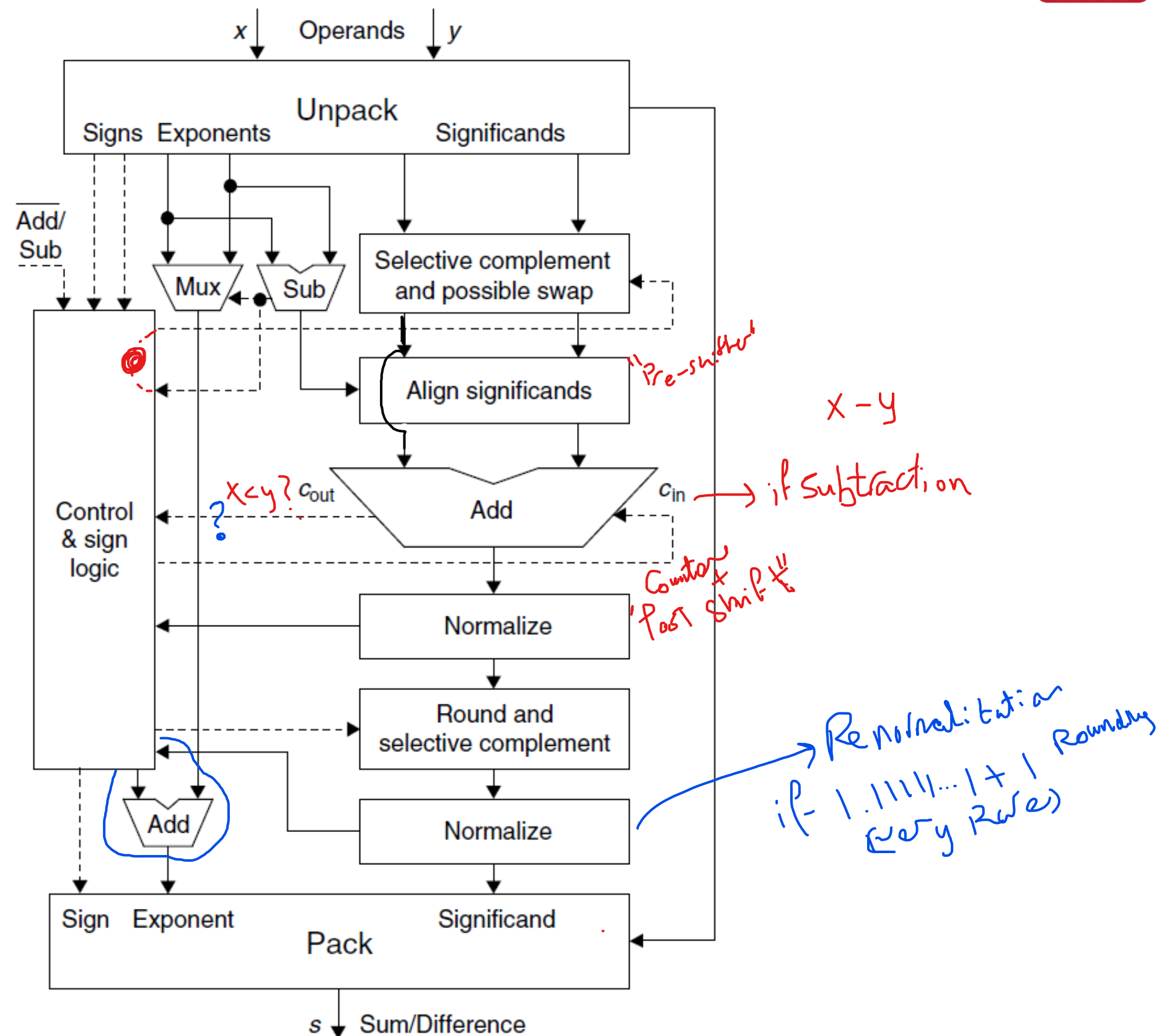


How addition works:

Normally, the two operands will have different exponents. Therefore, to perform the addition operation, the 2 operands must be **aligned**. This is done by shifting the operand with the lower exponent right, until its exponent reaches the larger exponent.

$$\begin{aligned}(\pm s_1 \times b^{e_1}) + (\pm s_2 \times b^{e_2}) &= (\pm s_1 \times b^{e_1}) + \left(\frac{\pm s_2}{b^{e_1 - e_2}} \times b^{e_1} \right) \\ &= \left(\pm s_1 \pm \frac{s_2}{b^{e_1 - e_2}} \right) \times b^{e_1} = \pm s \times b^e\end{aligned}$$

IEEE ASUSC
Solid-State
Circuits Society™

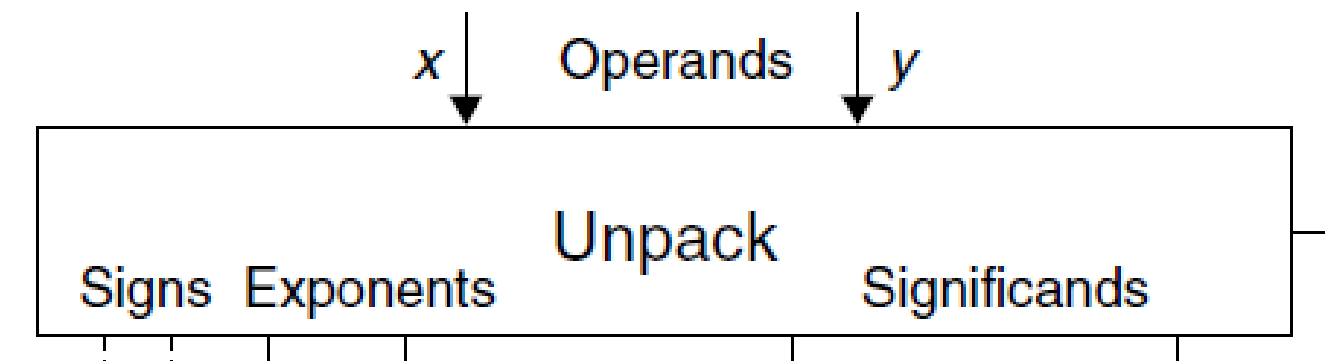


Floating-Point Adders/Subtractors

Unpack block:

This block is responsible for separating the sign, exponent, and significand for each operand, and reinstating the hidden 1 + sign bit.

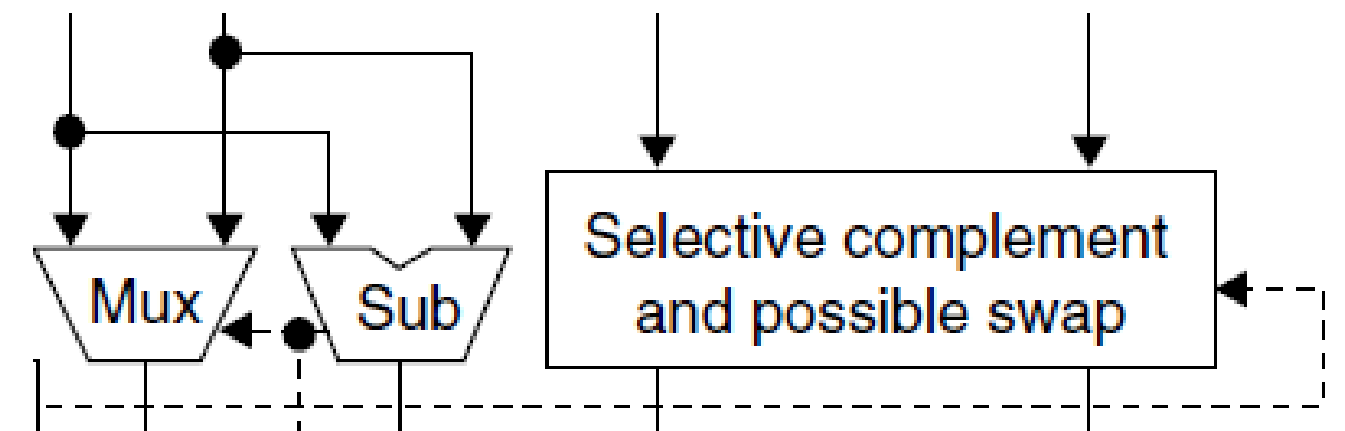
Testing for special operands and exceptions.



Floating-Point Adders/Subtractors

SCPS block:

This block handles multiple operations. First, it subtracts the exponents of the 2 operands and sends the result as the required shift to the alignment unit. The sign of the result determines which exponent is larger and sends it to the control unit to compute the final exponent.

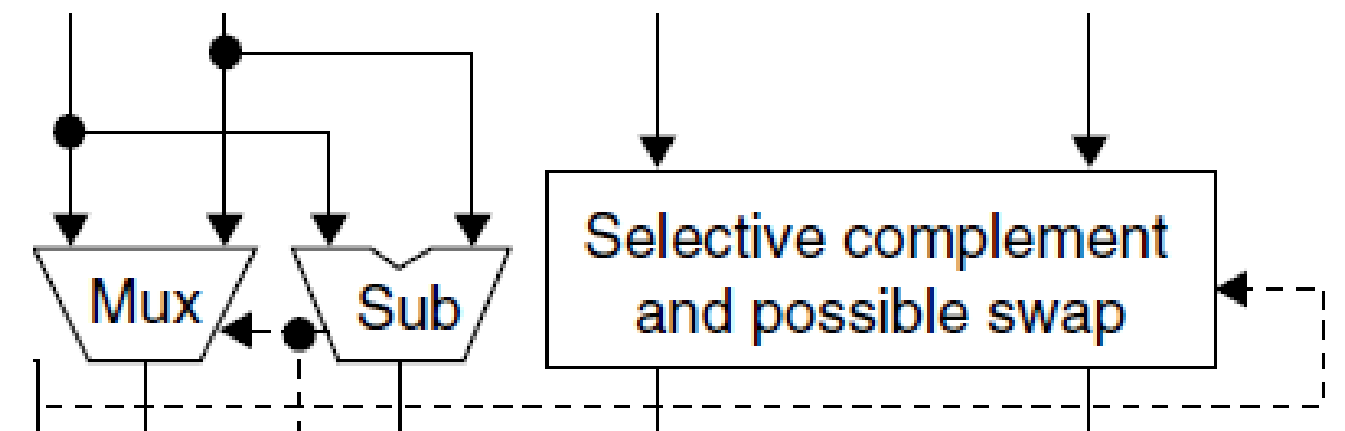


Floating-Point Adders/Subtractors

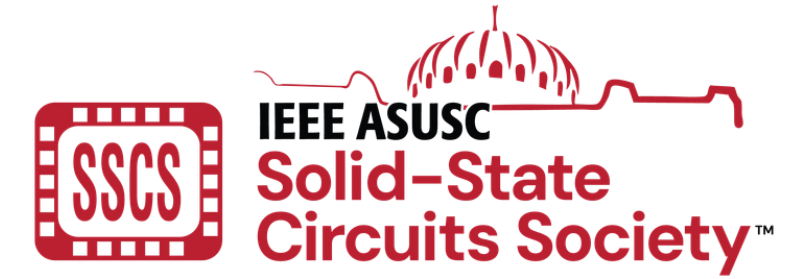
SCPS block:

The sign of the result of the difference of the two exponents has another function, where it decides whether to swap the 2 operands or not. But why would I need that?

Two operations must be performed on the operands before addition: they must be pre-shifted/aligned, and they must be complemented if the operands are to be subtracted.



Floating-Point Adders/Subtractors



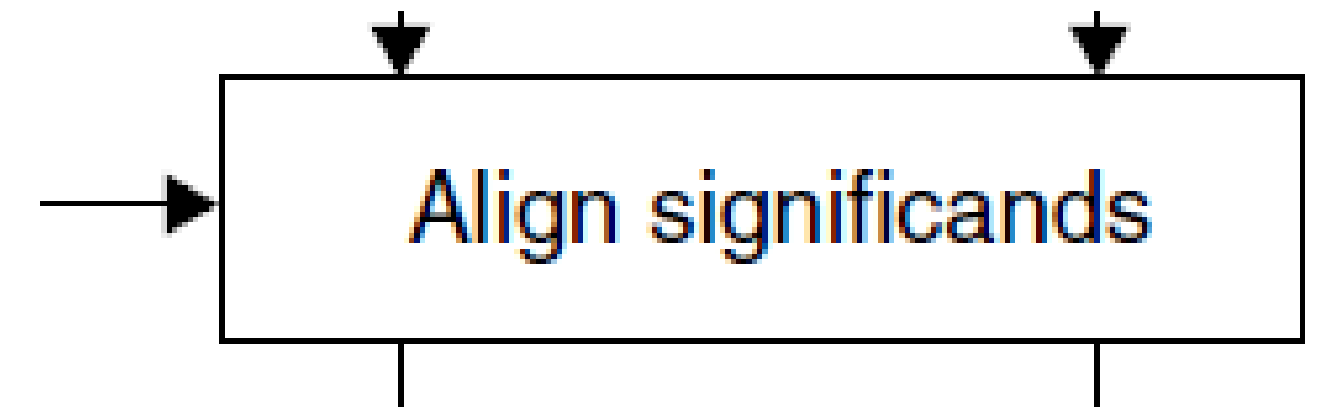
SCPS block:

To reduce the critical path delay, each operand is assigned one of the two required operations. Since the final result's sign is determined separately in the Control Unit, it is irrelevant which operand is complemented. What does matter, however, is which operand passes through the shifter. Because the shifter lies in the path of only one operand (e.g., the first operand), we may need to swap the operands. This ensures that the operand with the larger exponent is the one being shifted, preserving correctness while maintaining the shortest possible critical path.

Floating-Point Adders/Subtractors

Pre-shifter:

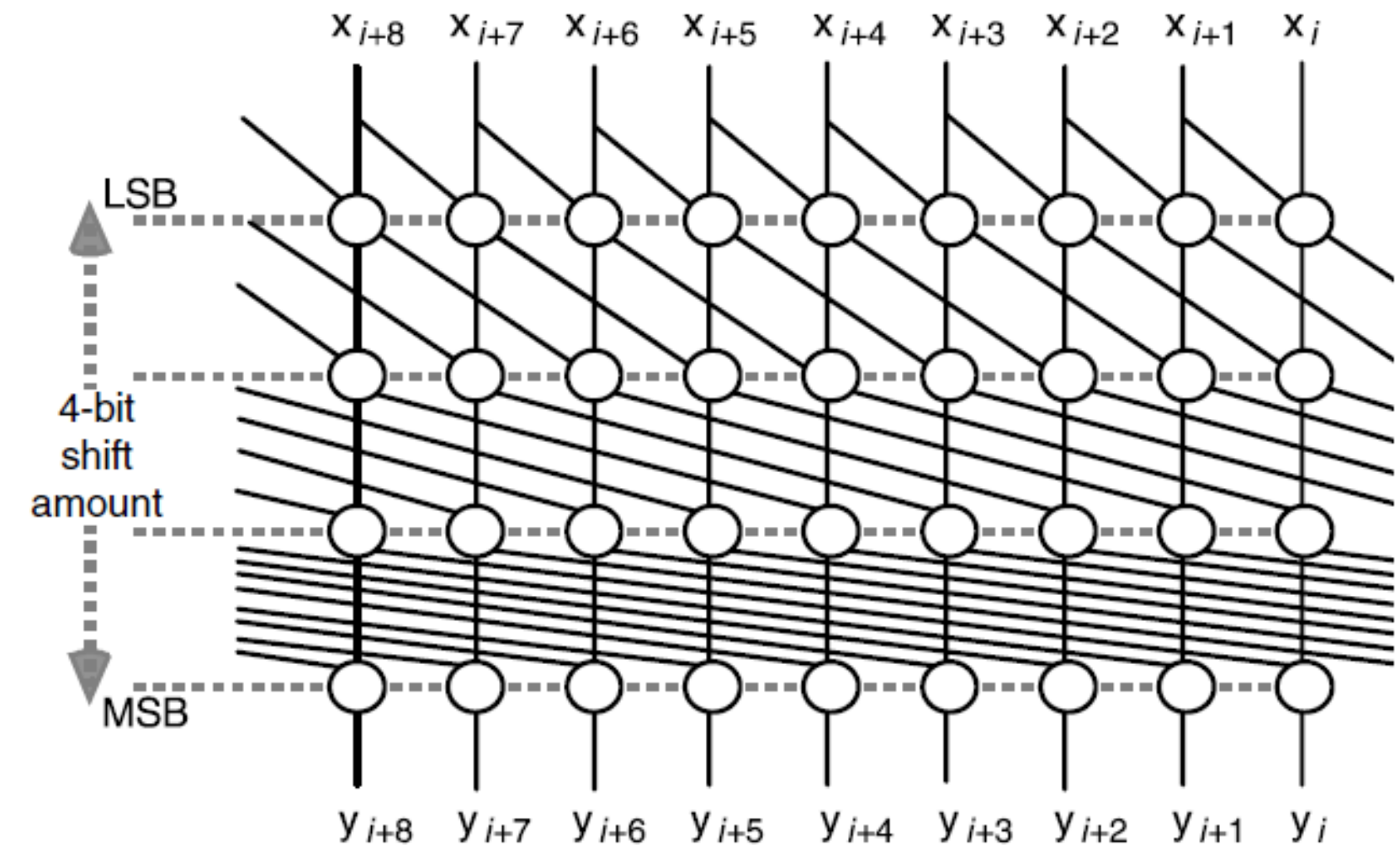
As discussed before, to correctly perform the addition/subtraction operation, the two operands must have the same exponents. Therefore, the role of this block is to shift the significand of the operand with the small exponent as required.



Floating-Point Adders/Subtractors

Pre-shifter:

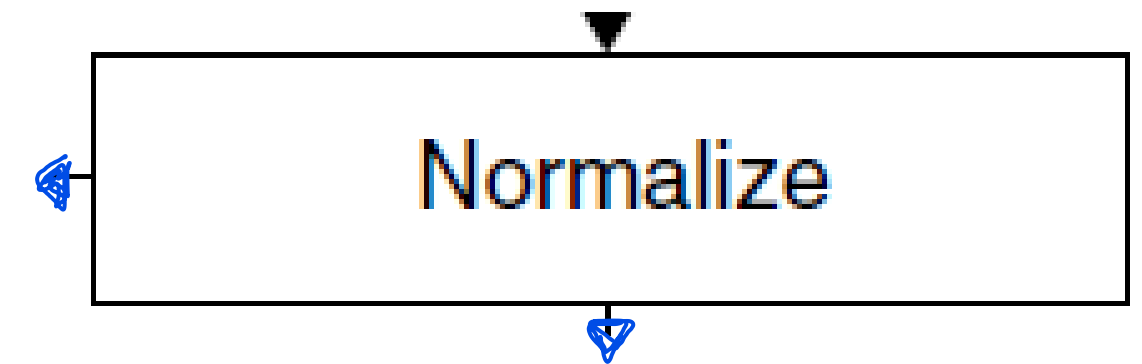
This can be achieved using a shifter like the one illustrated here, which has a delay complexity of $O(\log(k))$ and uses relatively simple hardware.



Floating-Point Adders/Subtractors

Post-shifter:

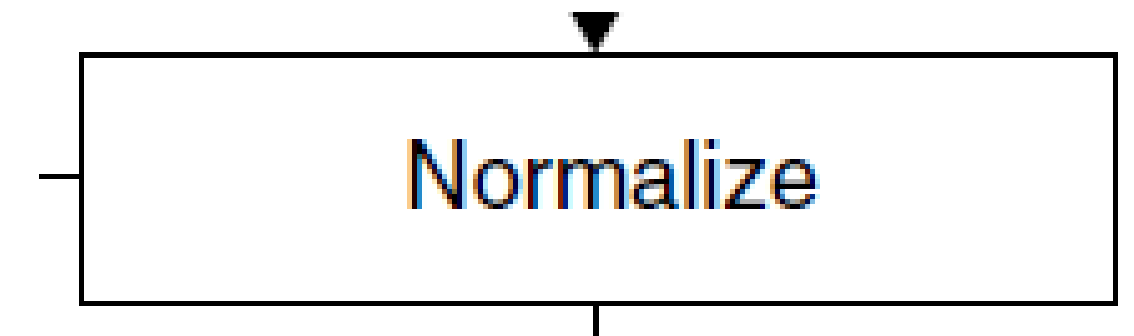
Imagine this: the two operands are comparable in magnitude, and you subtract the 2 significands. Naturally, the output will be very small. Since the representation of the significand must be 1.M, the result of the CPA must be **shifted left** to achieve this condition, and the shift amount will be accounted for in the exponent.



Floating-Point Adders/Subtractors

Post-shifter:

There is also another side to this coin: if the inputs are large and added to each other, overflow can occur. Therefore, the Post-shifter must shift the significand once to the right, and account for the shift amount in the final result's exponent.



Floating-Point Adders/Subtractors

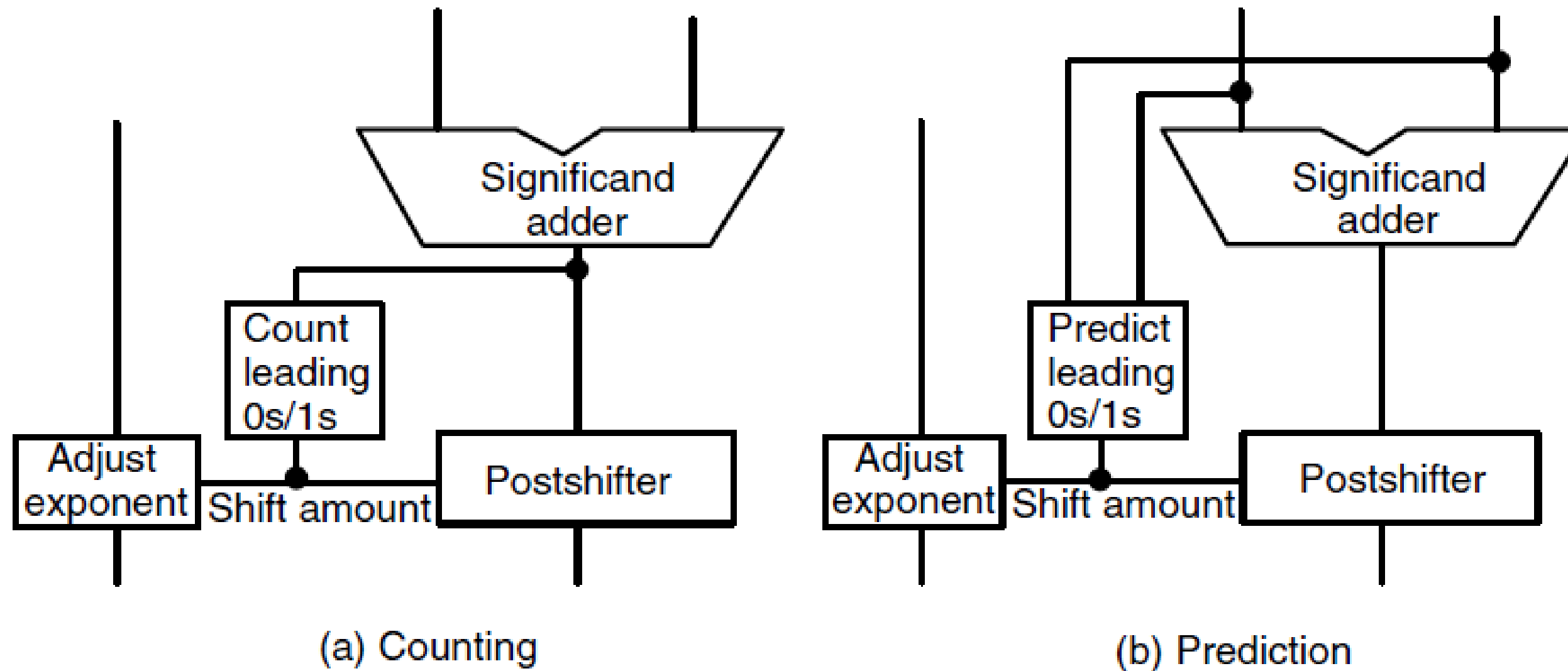
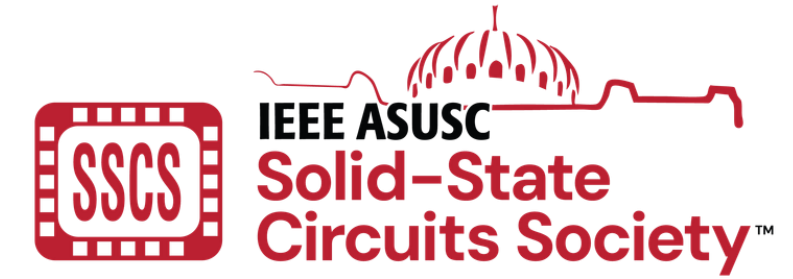


Figure 18.4 Leading zeros/ones counting versus prediction.

Floating-Point Adders/Subtractors



Post-shifter:

Instead of waiting for the CPA to output the sum/difference of the operands, we can try to predict the number of leading zeros/ones using a simple trick: searching for the patterns of the generate, propagate, and annihilate signals that will lead to leading zeros/ones. How?

Floating-Point Adders/Subtractors

Propagating

Post-shifter:

As shown, if the propagate signals from bit 0 until some bit $-i+1 = 1$, and if there is a generate signal at bit $-i$, followed by annihilate signals until bit $-j+1$, j leading zeros will be generated if the carry out of bit $-j$ is zero.

$$p_0 = p_{-1} = p_{-2} = \cdots = p_{-i+1} = 1$$
$$p_{-i} = 0 \quad (\text{i.e., } g_{-i} = 1 \text{ or } a_{-i} = 1)$$

In case $g_{-i} = 1$, let j be the smallest index such that:

$$g_{-i} = a_{-i-1} = a_{-i-2} = \cdots = a_{-j+1} = 1$$
$$a_{-j} = 0 \quad (\text{i.e., } g_{-j} = 1 \text{ or } p_{-j} = 1)$$

Floating-Point Adders/Subtractors

Predicting

Post-shifter:

Still with the same number of propagate signals until bit $-i+1$, but this time, there is an annihilate signal at bit $-i$, followed by generate signals until bit $-j+1$, j leading ones will be generated if the carry out of bit j is 1.

In case $a_{-i} = 1$, let j be the smallest index such that

$$a_{-i} = g_{-i-1} = g_{-i-2} = \cdots = g_{-j+1} = 1$$
$$g_{-j} = 0 \quad (\text{i.e., } p_{-j} = 1 \text{ or } a_{-j} = 1)$$

Floating-Point Adders/Subtractors

Counting

Post-shifter:

The next stage is a Priority Encoder tree that is used to detect the number of leading zeros/ones and returns the index of the leading one/zero.

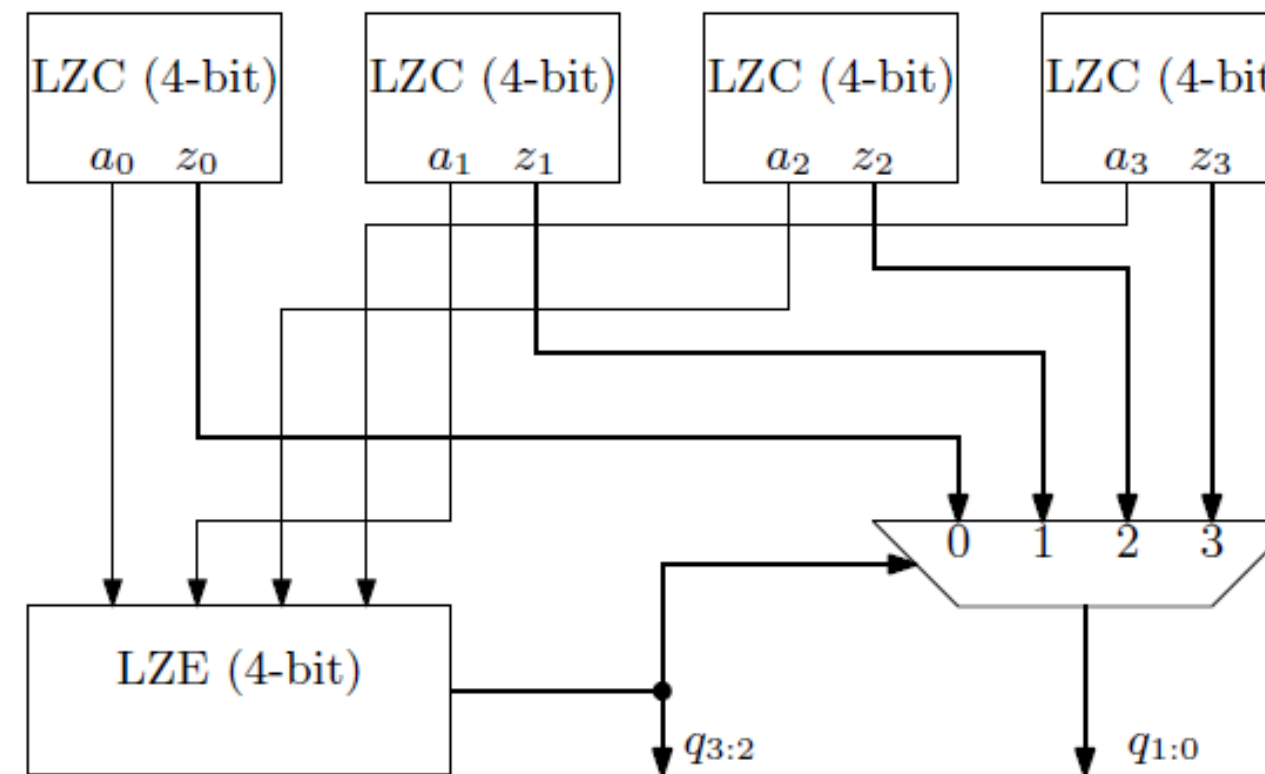


Fig. 12.7 A basic architecture for 16-bit leading zero counter

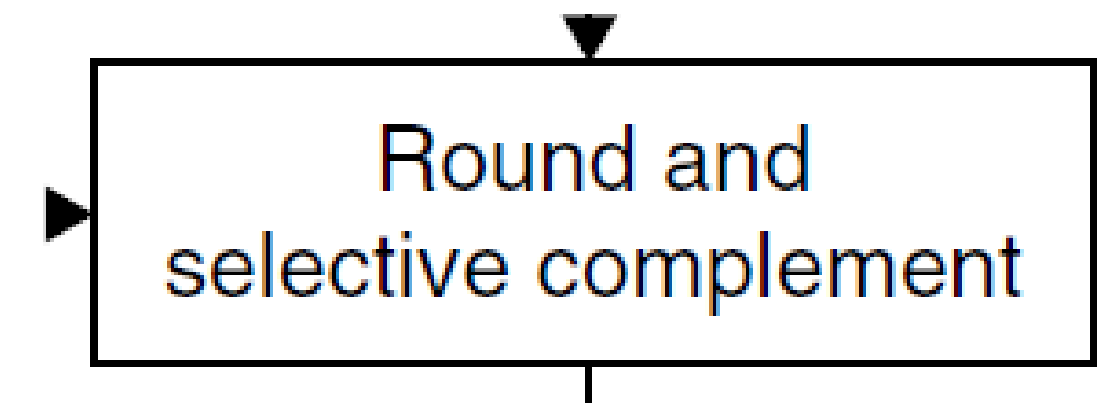
one more level

Floating-Point Adders/Subtractors

Rounding:

When shifting right for alignment, where will the shifted bits go?

First thought is just to let them be and double the width of the adder, maintaining full precision, but we will still have to fit the significand in 23 bits, thus rounding is necessary anyway.



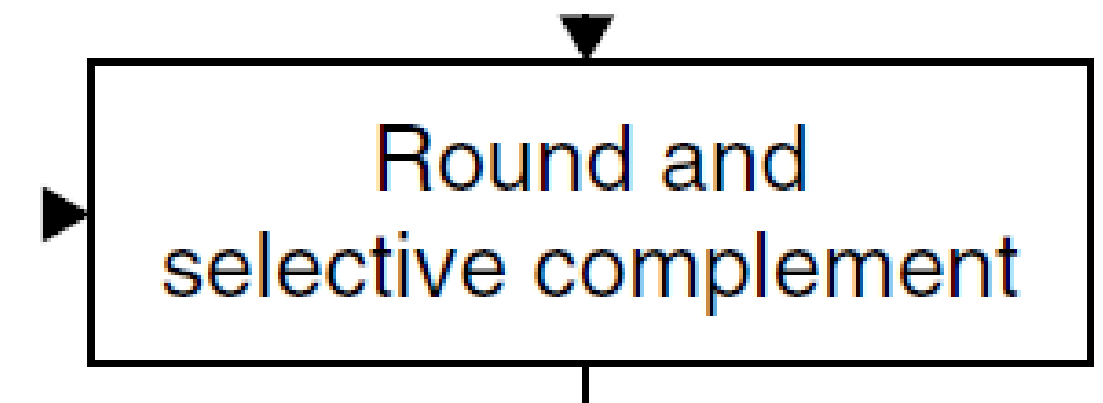
Floating-Point Adders/Subtractors

Rounding:

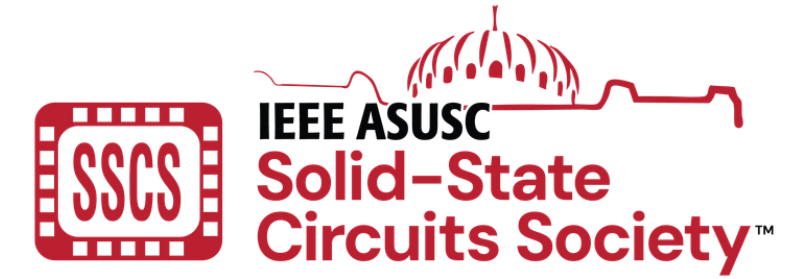
It was found that we did not need to keep all the bits; only three extra bits are needed to obtain reliable rounding.

These three bits are:

- Guard bit
- Round bit
- Sticky bit



Floating-Point Adders/Subtractors

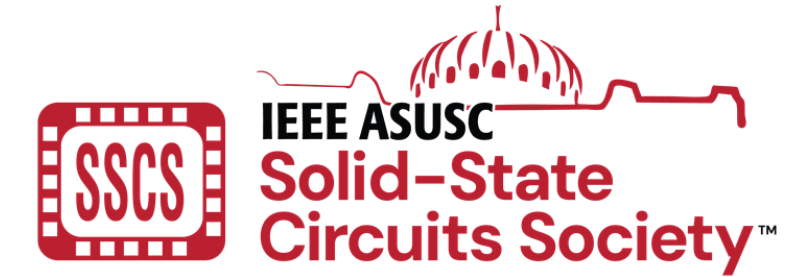


Rounding:

For alignment right shifts of 2 bits or more, the shifted significand will have a magnitude in $[0, 1/2)$.

Since the magnitude of the unshifted significand is in $[1, 2)$, the difference of the aligned significands will have a magnitude in $[1/2, 2)$. Thus, in this latter case, the normalization left shift will be by at most 1 bit, and G is still adequate to protect us against loss of precision.

Floating-Point Adders/Subtractors

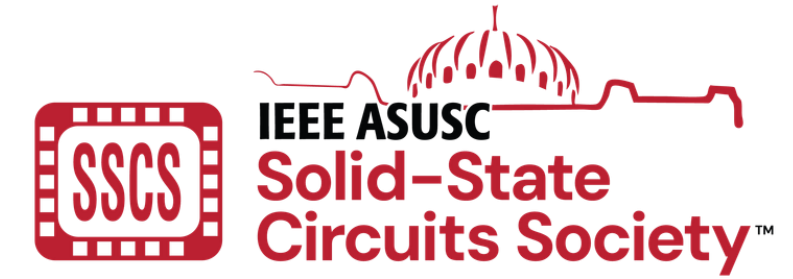


Rounding:

In case a normalization left shift actually takes place, the round bit is needed for determining whether to round the resulting significand down ($R = 0$, discarded part $< \text{ulp}/2$) or up ($R = 1$, discarded part $\geq \text{ulp}/2$). All that remains is to establish whether the discarded part is exactly equal to $\text{ulp}/2$.

Before postshifting (z)	$\cdots$	z_{-l+1}	z_{-l}		G	R	S
1-bit normalizing right-shift	$\cdots$	z_{-l+2}	z_{-l+1}		z_{-l}	G	$R \vee S$
1-bit normalizing left-shift	$\cdots$	z_{-l}	G		R	S	0
After normalization (Z)	$\cdots$	Z_{-l+1}	Z_{-l}		Z_{-l-1}	Z_{-l-2}	Z_{-l-3}

Floating-Point Adders/Subtractors

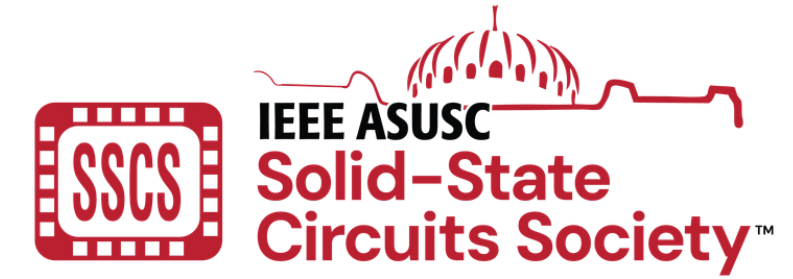


Rounding:

This information is needed in some rounding schemes, and providing it is the role of the sticky bit, which is set to the logical OR of all the bits that are shifted through it. Thus, following an alignment right shift of 7 bits, say, the sticky bit will be set to the logical OR of the 5 bits that move past G and R.

Doing nothing	if $Z_{l-1} = 0$ or $Z_l = Z_{l-2} = Z_{l-3} = 0$
Adding $ulp = 2^{-l}$	otherwise

Floating-Point Adders/Subtractors



Rounding:

As discussed earlier in this section, a pre-shift of 2 bits or more rules out the possibility of requiring a multiple-bit post-shift to remove leading 0s or 1s. Very fast floating-point adder designs take advantage of this property in a dual-path arrangement. When the exponents differ by no more than 1 (the inputs are close to each other in order of magnitude), the significands are routed to the near data path that contains a leading 0s/1s predictor and a full post-shifter; in fact, one can even avoid rounding in this path.

Floating-Point Adders/Subtractors

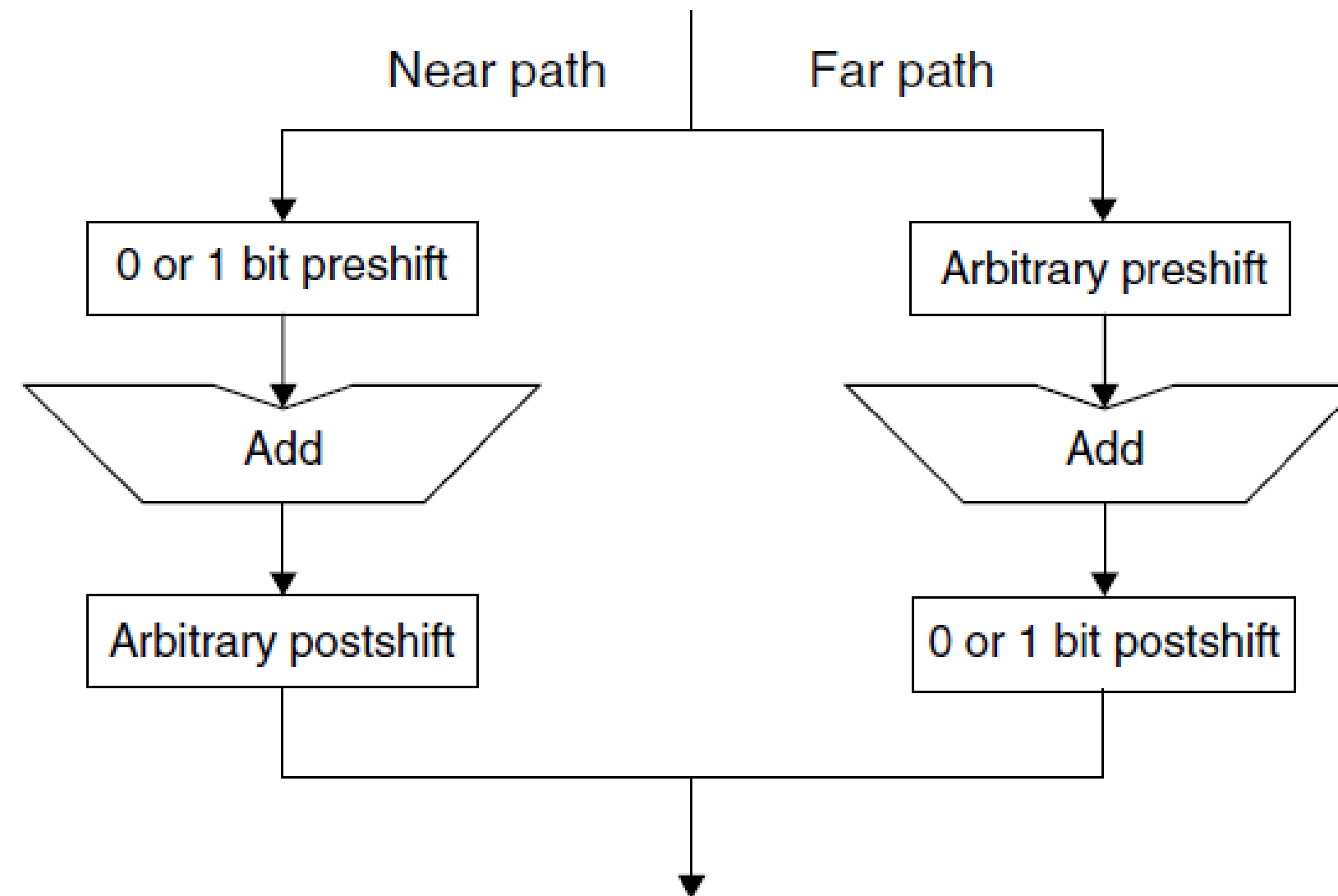
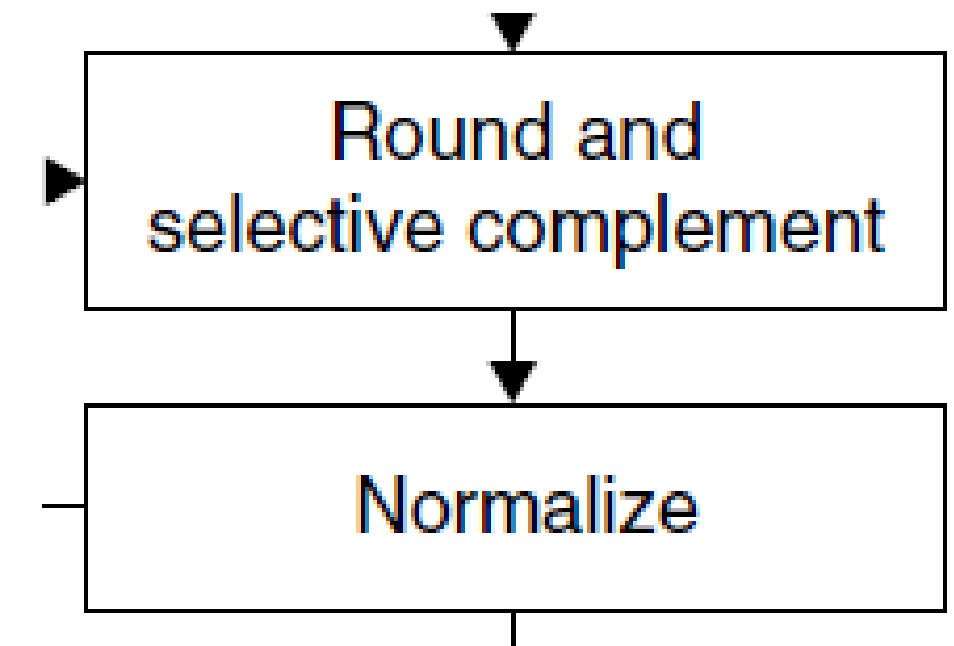


Figure 18.5 Conceptual view of significantand handling in a dual-path floating-point adder.

Floating-Point Adders/Subtractors

2nd Normalize Block:

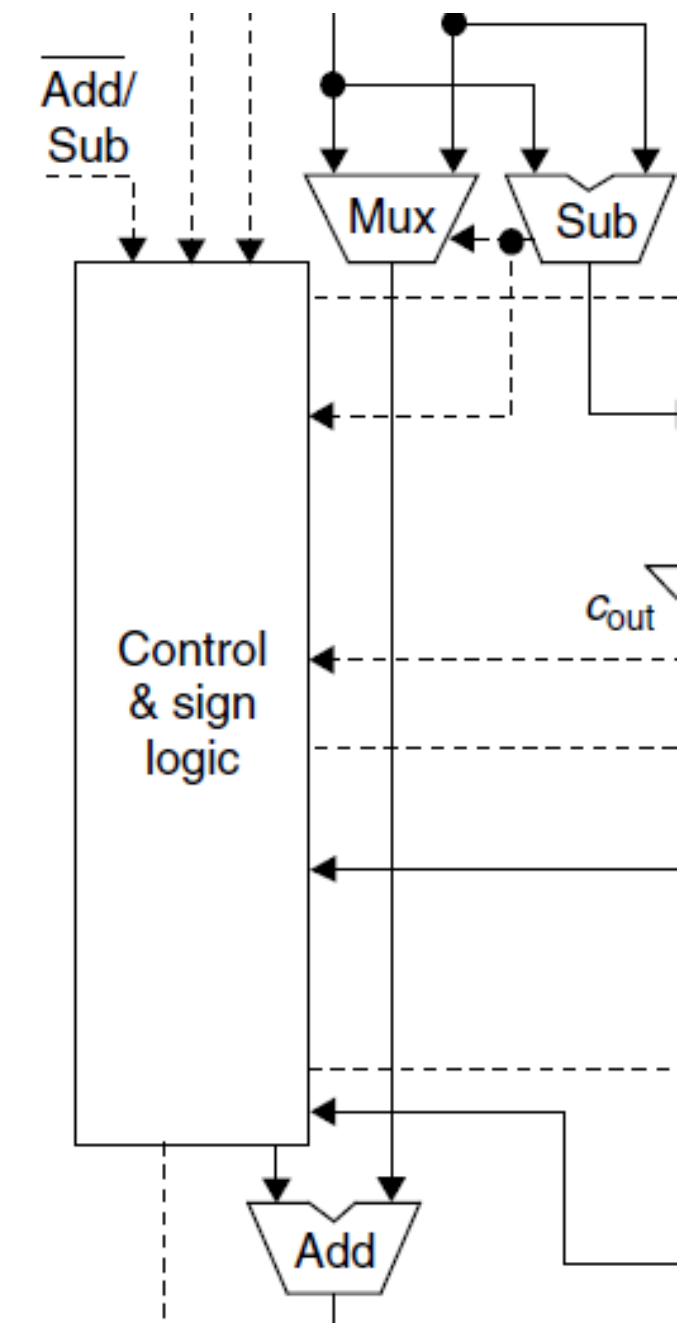
After rounding, there is a chance that overflow occurs; thus, we need another normalization block to put the significand in the appropriate range and increment the exponent.



Floating-Point Adders/Subtractors

Control Unit:

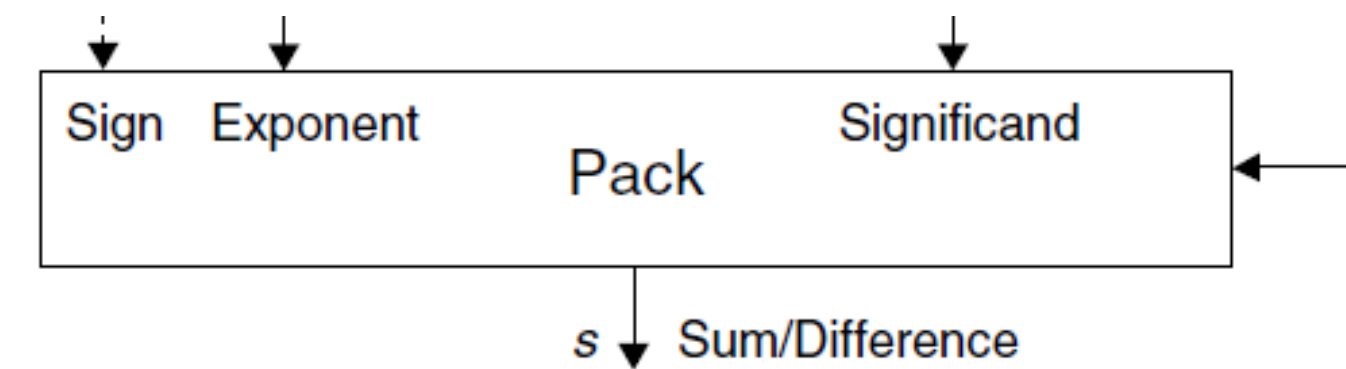
Responsible for handling the operation type, whether addition or subtraction, handling special cases, and handle the sign logic.



Floating-Point Adders/Subtractors

Pack Block:

Responsible for combining the sign, exponent, and the significand for the result and removing the hidden one. If there is a special outcome, it is responsible for outputting the value corresponding to the case.

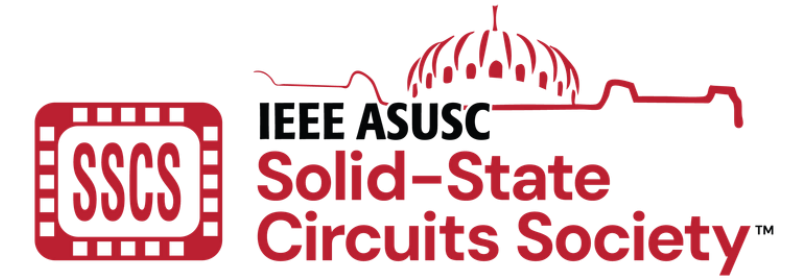


4

Floating-Point **Multipliers/Dividers**



Floating-Point Multipliers/Dividers



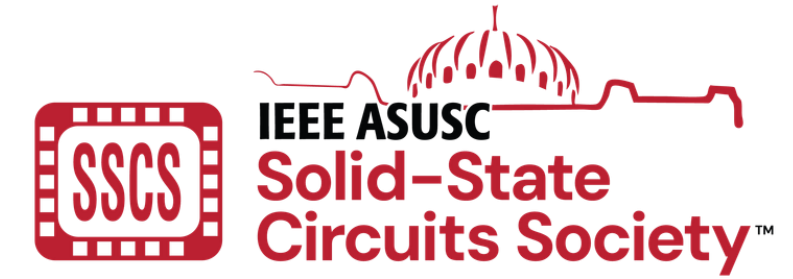
How multiplication works:

Multiplication is more straightforward than addition, since you are just required to multiply the significands and add the exponents.

The only catch is to remember that the exponents are actually **biased**. Therefore, adding the exponents would add the bias twice; thus, one bias must be subtracted from the sum.

$$(\pm s1 \times b^{e1}) \times (\pm s2 \times b^{e2}) = \pm (s1 \times s2) \times b^{e1+e2}$$

Floating-Point Multipliers/Dividers



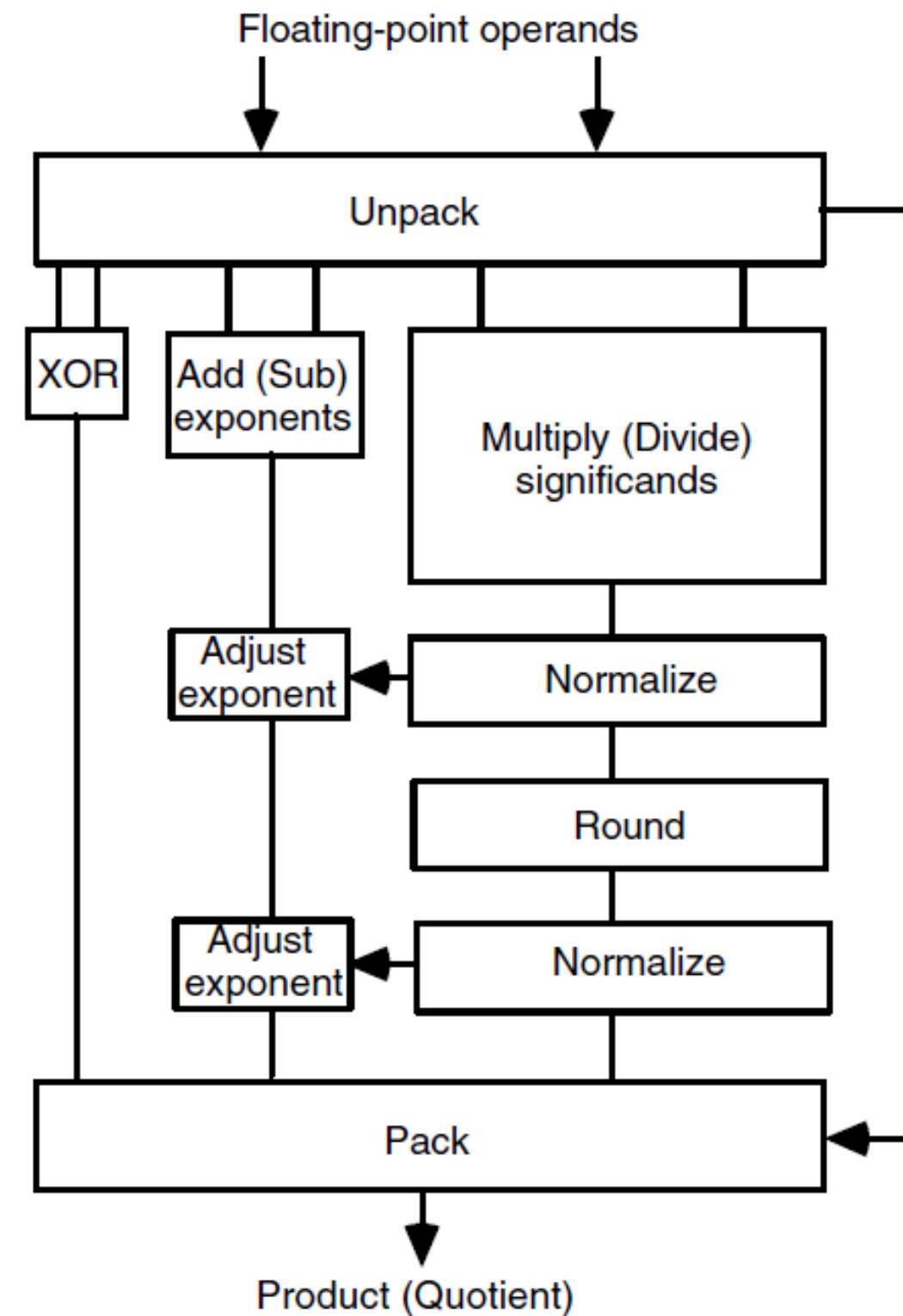
How division works:

Division is almost the same hardware as the multiplier; the only difference is that we are dividing the significands and subtracting the exponents.

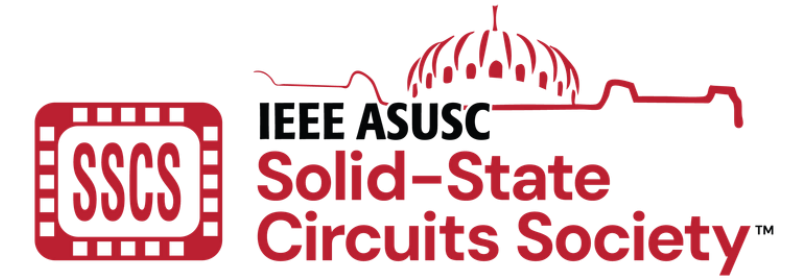
Like before, subtracting the exponents removes the bias; thus, it must be added to the difference of the exponents at the end of the operation.

$$\frac{\pm s1 \times b^{e1}}{\pm s2 \times b^{e2}} = \pm \frac{s1}{s2} \times b^{e1-e2}$$

Floating-Point Multipliers/Dividers



Floating-Point Multipliers/Dividers



Rounding:

Question: What are the extra bits needed in Floating-point Multiplication and Division for reliable accuracy?



Fused **Multiply-Add**



Fused Multiply-Add

Computing $p = ax+b$ is a natural operation for direct hardware implementation, once we move beyond the four basic arithmetic operations.

Fused multiply-add is useful in two very common computation sequences, namely, polynomial evaluation and vector dot product.

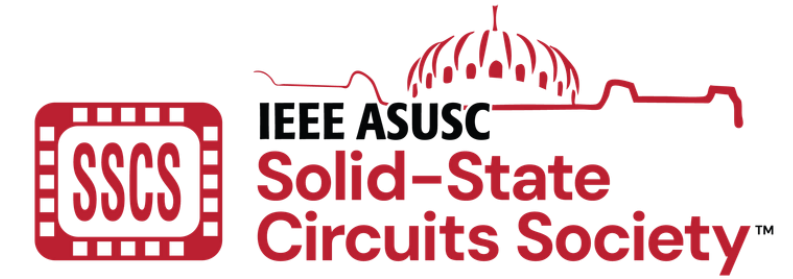
$$s := sz + c(j) \quad \text{for } j \text{ from } n-1 \text{ down to } 0$$

where $c^{(j)}$ is a coefficient of the polynomial $f(z) = c^{(n-1)}z^{n-1} + c^{(n-2)}z^{n-2} + \dots + c^{(1)}z + c^{(0)}$ and the running total s is initialized to 0. Similarly, the dot product of the n -vectors u and v , with their elements indexed from 0 to $n-1$, can be evaluated via

$$s := s + u^{(j)}v^{(j)} \quad \text{for } j \text{ from } 0 \text{ upto } n-1$$

beginning with $s = 0$.

Fused **Multiply-Add**



General rule: if something is constant in the system, optimize it as much as possible. Based on that, my system will repeatedly utilize the multiply-add operation; therefore, instead of opting for a simple solution, such as cascading a multiplier then an adder, we could use some of the optimization we have learned to accelerate the operation.

Based on that conclusion, we will remove redundant operations and utilize reduction trees to handle multiplication and addition in one swift motion.

Removing redundant operations means rounding and normalizing only once at the very end of the operation.

Fused Multiply-Add

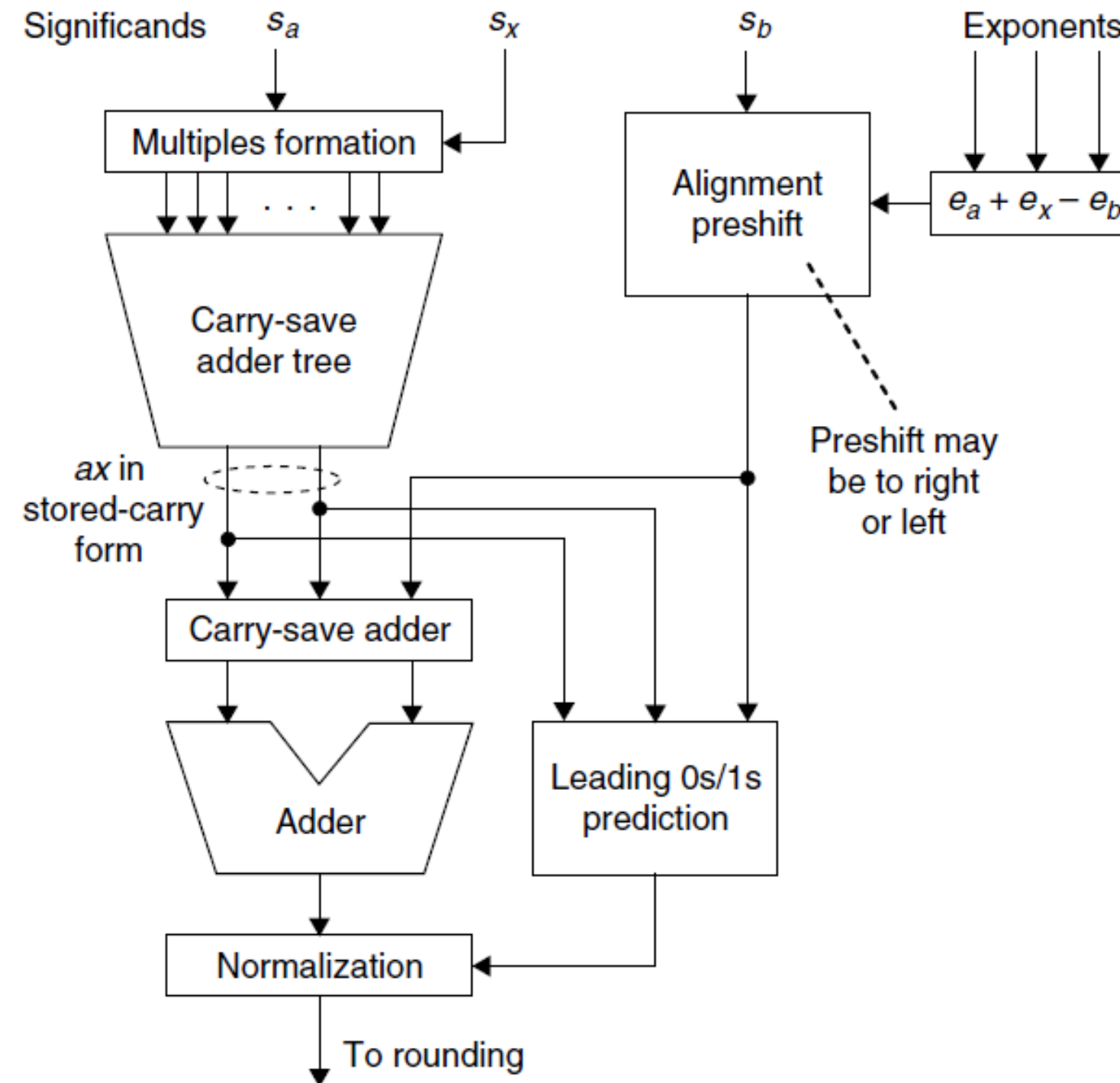


Figure 18.7 Block diagram of a fast FMA unit.

6

Logarithmic Arithmetic Unit



Logarithmic Arithmetic Unit

Why use a Logarithmic arithmetic unit anyway?

Because it accelerates multiplication, division, and especially the square root operation. Since these operations are reduced to addition, subtraction, and shifting right, respectively.

$$(Sx, Lx) = (\text{sign}(x), \log_2 |x|)$$

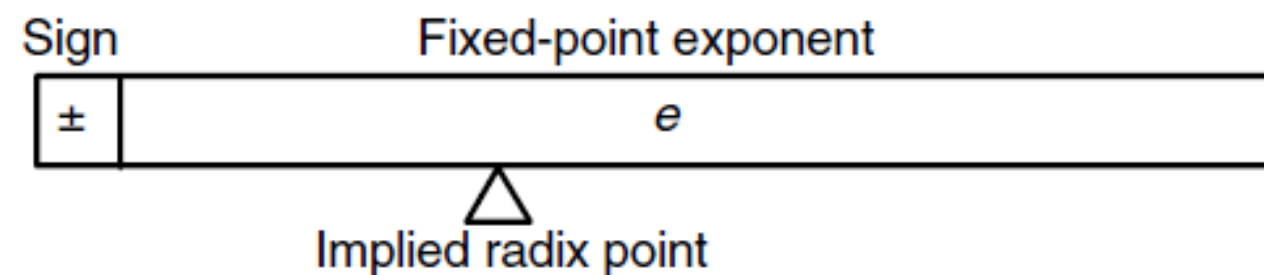


Figure 17.13 Logarithmic number representation with sign and fixed-point exponent.

Logarithmic Arithmetic Unit

Now, we handled multiplication and division, but what about addition and subtraction?

$$(Sx, Lx) \pm (Sy, Ly) = (Sz, Lz)$$

for logarithmic operands and assume $x > y > 0$ (other cases are similar). Then

$$\begin{aligned} Lz = \log z &= \log(x \pm y) = \log(x(1 \pm y/x)) \\ &= \log x + \log(1 \pm y/x) \end{aligned}$$

Note that $\log x$ is known and $\log(y/x)$ is easily computed as $\Delta = -(\log x - \log y)$. Given Δ , the term

$$\log(1 \pm y/x) = \log(1 \pm \log^{-1} \Delta)$$

is easily obtained by table lookup (two tables, ϕ^+ and ϕ^- , are needed). Hence, addition and subtraction of logarithmic numbers can be based on the following computations:

$$\log(x + y) = \log x + \phi^+(\Delta)$$

$$\log(x - y) = \log x + \phi^-(\Delta)$$

Logarithmic Arithmetic Unit

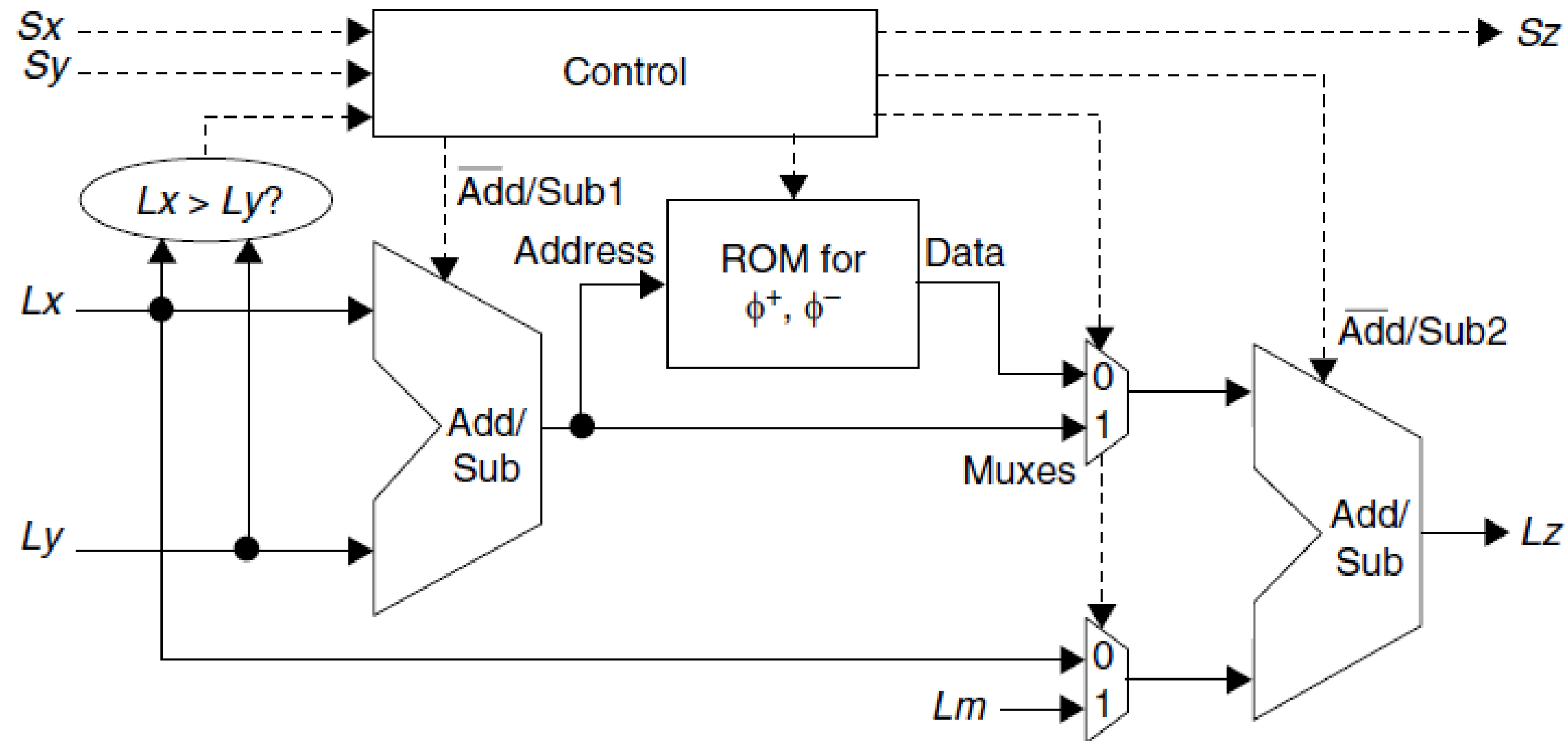
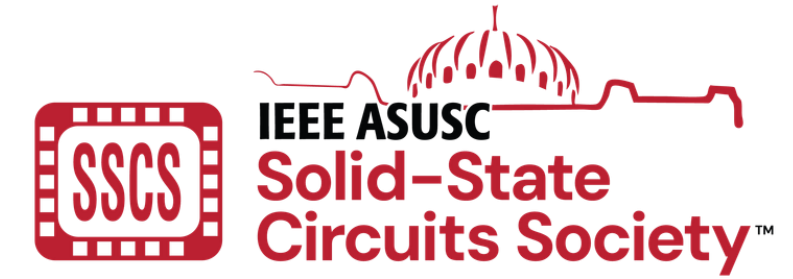


Figure 18.8 Arithmetic unit for a logarithmic number system.

References



- Eng. Ahmed Abdelsalam Slides, STMicroelectronics
- Computer Arithmetic: Algorithms and Hardware Designs, Behrooz Parhami
- Advanced Digital System Design: A Practical Guide to Verilog-Based FPGA and ASIC Implementation, Shirshendu Roy

Thank You for
Your Attention!